ECE 220 Computer Systems & Programming

Lecture 16 — Dynamic Memory Allocation

October 20, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

One Function, Any Roster

One function reads a roster file — 5 students or 400 — and hands the records back to whoever called it.

? `read_roster(`*filename*`)`

it has to hand back **the records** and **how many** there are

One Function, Any Roster

One function reads a roster file — 5 students or 400 — and hands the records back to whoever called it.

? `read_roster(`*filename*`)`

it has to hand back **the records** and **how many** there are

Where do those records live once `read_roster` returns?

Two Rosters, One Function

```
student *read_roster(char *filename, int *n) {
    student roster[200];   /* 200: fixed when compiled */
    FILE *fp = fopen(filename, "r");
    int i;
    if (fp == NULL)
        return NULL;
    fscanf(fp, "%d", n);
    for (i = 0; i < *n; i++)
        fscanf(fp, "%s %d %f", roster[i].Name,
               &roster[i].UIN, &roster[i].GPA);
    fclose(fp);
    return roster;          /* address of a local */
}
    sec_a = read_roster("roster.txt", &n_a);
    sec_b = read_roster("roster_b.txt", &n_b);
```

```
gcc: warning:  address of stack memory associated
with local variable 'roster' returned
```

Two Rosters, One Function

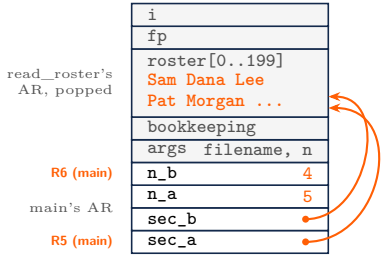
```
student *read_roster(char *filename, int *n) {
    student roster[200];   /* 200: fixed when compiled */
    FILE *fp = fopen(filename, "r");
    int i;
    if (fp == NULL)
        return NULL;
    fscanf(fp, "%d", n);
    for (i = 0; i < *n; i++)
        fscanf(fp, "%s %d %f", roster[i].Name,
               &roster[i].UIN, &roster[i].GPA);
    fclose(fp);
    return roster;          /* address of a local */
}
    sec_a = read_roster("roster.txt", &n_a);
    sec_b = read_roster("roster_b.txt", &n_b);
```

gcc: warning: address of stack memory associated with local variable 'roster' returned

`roster.txt` holds 5 students, `roster_b.txt` holds 4. Section A prints

Sam, Dana, Lee, Pat, Morgan

Draw the run-time stack (RTS) after the second call returns:



`roster` lived in `read_roster`'s activation record (AR). The AR was popped, and the second call built its own in the same slots.

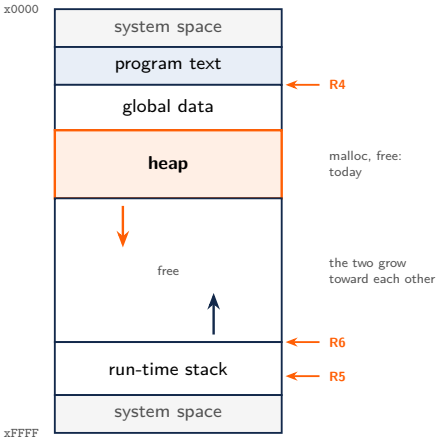
Today's Three Questions

The run-time stack lends a function memory and takes it back when the function returns.

1. Where does memory come from when its size is known only at run time, and the block must outlive the function that asked for it?
2. How does a program ask for a block, and how does it give it back?
3. What goes wrong when it forgets to give it back, gives it back twice, or keeps using it afterwards?

Lecture 5's memory map had a box marked "heap (Lecture 16)". Today fills it in.

Review: Where Things Live



Lower addresses at the top, as in Lecture 8.

The `student` struct: `Name[100]`, `UIN`, `GPA`;
`sizeof(student)` is 108 bytes.

- `student ece220[200];` outside `main`: 21600 bytes in **global data**, alive **the whole run**
- `student s1;` in `main`: 108 bytes in **main's AR**, popped when **main returns**
- `student *s_ptr = ece220;` in `main`: the pointer is in **main's AR**; the address it holds is in **global data**
- `student roster[200];` in `read_roster`: 21600 bytes in **read_roster's AR**, popped at return
- Today: a block from `malloc` is in the **heap** until the program frees it

The heap and the stack grow toward each other through the free region between them.

"Static" vs Dynamic Allocation

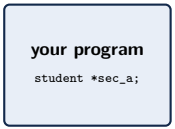
"Static" here means fixed by a declaration. It covers both storage classes: static (globals) and automatic (locals).

	"Static"	Dynamic
Mechanism	a declaration; the compiler assigns the home: a label, or an offset from R5	a call to `malloc`, `calloc` or `realloc`; the program keeps the address in a pointer
Lifetime	globals: the whole run; locals: from entry into the block to its exit	from the call until `free`; the program decides
Location	global data section, or the run-time stack	heap
Size	fixed when the program is compiled	chosen while it runs; changed later by `realloc`

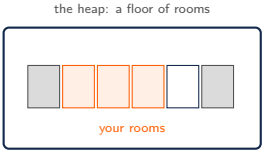
A dynamic block has no name. The only way back to it is the address `malloc` returned.

The Heap as a Hotel

Three parties: your program, the allocator at the front desk, and the rooms themselves.

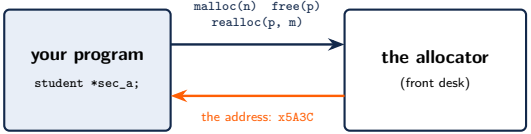


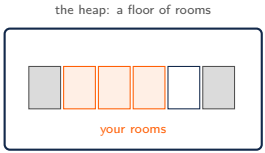




The Heap as a Hotel

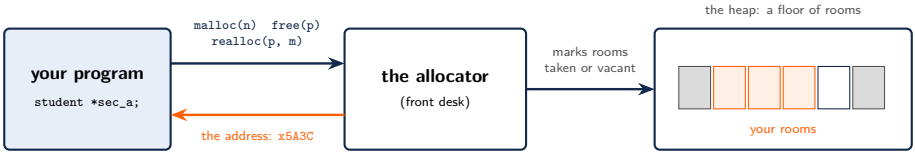
Three parties: your program, the allocator at the front desk, and the rooms themselves.





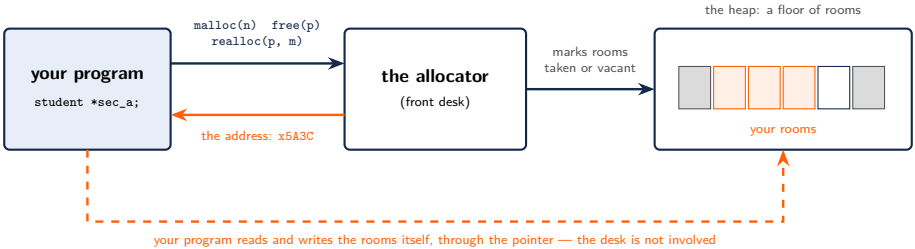
The Heap as a Hotel

Three parties: your program, the allocator at the front desk, and the rooms themselves.



The Heap as a Hotel

Three parties: your program, the allocator at the front desk, and the rooms themselves.



The desk hears from you only at `malloc`, `free` and `realloc`. In between, that one address is your only way back into the rooms.

malloc and free

```
ptr = (int *)malloc(sizeof(int));
if (ptr == NULL) {
    printf("ERROR - malloc failure!\n");
    return 1;
}
*ptr = 10;
printf("*ptr = %d\n", *ptr);
free(ptr);
```

Output: `*ptr = 10`

An array of 10 ints, in one block:

`arr =` **`(int *)malloc(10 * sizeof(int))`**

Then `arr[i]` as for any array, and one `free(arr)` returns all **40** bytes.

`void free(void *ptr)`

- gives the whole block back; `ptr` must be an address that `malloc`, `calloc` or `realloc` returned

`void *malloc(size_t size)`

- asks the allocator for `size` contiguous bytes on the heap; `size_t` is an unsigned integer type
- returns the block's first address as a `void *`, an address with no type; the cast `(int *)` gives it one
- returns `NULL` when no block that large is free. Writing through `NULL` is a **segmentation fault**: the operating system stops a program that touches an address it does not own
- the bytes are **not initialized**

In-Class Problem: read_roster on the Heap

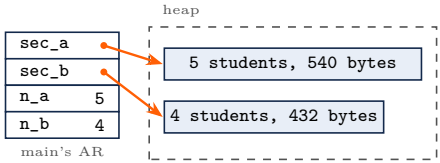
```
student *read_roster(char *filename, int *n) {
    student *roster;
    fscanf(fp, "%d", n);
    /* a block of *n students; NULL if none */
    roster = (student *)malloc(*n * sizeof(student));
    if (roster == NULL) {
        fclose(fp);
        return NULL;
    }
    for (i = 0; i < *n; i++)
        fscanf(fp, "%s %d %f", roster[i].Name,
               &roster[i].UIN, &roster[i].GPA);
    fclose(fp);
    return roster;   /* heap address: valid */
}
```

and in `main`:

```
    student *sec_a, *sec_b;
    int n_a, n_b;
    sec_a = read_roster("roster.txt", &n_a);
    sec_b = read_roster("roster_b.txt", &n_b);
```

- Bytes asked for, `*n * sizeof(student)`: 540 for Section A, 432 for B
- `main`, after printing both: `free(sec_a); free(sec_b);`
- Section A now prints Jamie, Alex, Robin, Casey, Morgan
- `ftranspose_heap.c`, the transpose, the same way: `m * n` ints once `m` and `n` are read; element (r, c) is `in_matrix[r * n + c]`

Draw `main`'s AR and the heap:



Losing a Block: Memory Leak

```
ptr = (int *)malloc(sizeof(int));
*ptr = 5;
ptr_2 = (int *)malloc(sizeof(int));
*ptr_2 = 6;
ptr = ptr_2;
printf("*ptr = %d, *ptr_2 = %d\n", *ptr, *ptr_2);
free(ptr_2);
```

Output: `*ptr = 6, *ptr_2 = 6`

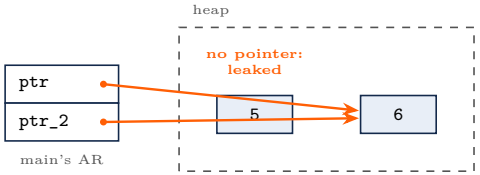
`leaks -atExit -- ./leak` reports **1** leak, of 16 bytes: the allocator's smallest block holds the 5

Add `free(ptr); free(ptr_2);` at the end instead:

the same block is freed twice; the 5 is still lost

A **double free**: freeing one address twice. The allocator detects it and stops the program; `double_free.c` ends with `Trace/BPT trap`.

Draw the heap after `ptr = ptr_2`:



A **memory leak**: a block that is still allocated but that no pointer in the program reaches. It stays taken until the program ends.

The fix: **free(ptr);** before `ptr = ptr_2;`

One free per malloc, through any pointer that still holds the address.

After free: the Dangling Pointer

```
ptr = (int *)malloc(sizeof(int));
if (ptr == NULL)
    return 1;
*ptr = 10;
free(ptr);
/* ptr dangles: the block is not ours */
printf("%d\n", *ptr);  /* undefined */
ptr = NULL;            /* the habit */
free(ptr);             /* does nothing */
printf("%d\n", *ptr);  /* seg fault */
```

One run, on a Mac:

```
7
Segmentation fault: 11
```

The second `printf` is a **segmentation fault**: the operating system stops the program at that line.

- After `free(ptr)`, `ptr` still holds the address: a **dangling pointer**. The block is the allocator's again, and may already be in use.
- Reading or writing through it is **undefined behavior**: here the allocator had written in the block and the 10 was gone. Another run or machine prints something else.
- `free(NULL)` does nothing, so a pointer set to `NULL` after `free` is safe to free again
- The opener's `sec_a` was dangling too: the AR that held its block had been popped

Set a pointer to NULL after free. A NULL dereference stops the program where the bug is.

calloc: a Zeroed Block

```
a = (int *)malloc(N * sizeof(int));
if (a == NULL)
    return 1;
for (i = 0; i < N; i++)
    a[i] = 7;
free(a);   /* the block goes back */

a = (int *)malloc(N * sizeof(int));
b = (int *)calloc(N, sizeof(int));
```

Then it prints the first five ints of each. On this Mac:

```
malloc: 7 7 7 7 7
calloc: 0 0 0 0 0
```

`N` is 50000. The 200,000-byte block came back as `free` left it. Small freed blocks on this Mac come back zeroed; the C standard promises nothing for `malloc`.

`void *calloc(size_t n_items, size_t item_size)`

- a block of **n_items * item_size** bytes, every byte set to zero
- otherwise as `malloc`: `NULL` on failure, one `free` for the whole block
- `char *ptr2 = (char *)calloc(100, sizeof(char));` gives **100** bytes of `'\0'`: already **an empty string**, ready for `strncpy`

Use calloc when the code reads from the block before it has written every byte.

realloc: Resize, and Maybe Move

```
ptr2 = (char *)calloc(100, sizeof(char));
if (ptr2 == NULL)
    return 1;
strncpy(ptr2, "Example using calloc", 100);
printf("%s, at %p\n", ptr2, (void *)ptr2);

ptr3 = (char *)realloc(ptr2, 200 * sizeof(char));
if (ptr3 == NULL) {
    free(ptr2);
    return 1;
}
printf("%s, at %p\n", ptr3, (void *)ptr3);
free(ptr3);
```

One run on a Mac; the addresses change from run to run:

```
Example using calloc, at 0x1033659c0
Example using calloc, at 0x103365aa0
```

`void *realloc(void *ptr, size_t size)`

- resizes the block at `ptr` to `size` bytes and returns its address
- room behind the block: it grows in place, same address
- no room: a new block of **200** bytes; the old **100** bytes are copied over; the old block is **freed** by realloc
- `ptr2` afterwards: **dangling**
- `free(ptr3)` releases **200** bytes. `free(ptr2)` as well? **No**: a double free
- `ptr == NULL`: same as **malloc**. `size == 0`: implementation specific

Keep the old pointer until realloc has succeeded: on failure the old block is still yours.

In-Class Problem: One Record, and Freeing Them All

`typedef struct studentStruct { char *NAME; int UIN; float GPA; } student;`
`sizeof(student)` = **16**: 8 + 4 + 4

```
/* NAME gets a 100-char block; set all 3 members */
void init_record(student *s) {
    s->NAME = (char *)malloc(100 * sizeof(char));
    if (s->NAME != NULL)
        strncpy(s->NAME, "To be set", 100);
    s->UIN = -1;
    s->GPA = 0.0;
}
```

```
/* free n NAME blocks, then the records */
void free_all(student *records, int n) {
    int i;
    for (i = 0; i < n; i++)
        free(records[i].NAME);
    free(records);
}
```

`free(records)` first would discard the 400 addresses, and every NAME block would leak.

`init_record` asks for **100** bytes. `init_record(&records[i])` passes **one address**, so the function fills record `i` in place.

In-Class Problem: 200 Records, Then 400

```c
int main() {
    student *records, *grown;
    int i;

    /* 1. a block of 200 records */
    records = (student *)malloc(200 * sizeof(student));
    if (records == NULL)
        return 1;
    /* 2. initialize all 200 */
    for (i = 0; i < 200; i++)
        init_record(&records[i]);
```

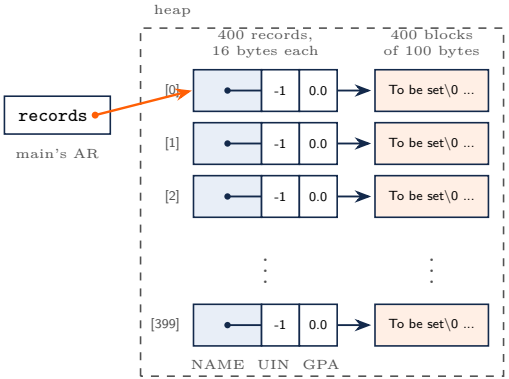
```c
    /* 3. grow to 400; initialize 200..399 */
    grown = (student *)realloc(records,
                               400 * sizeof(student));
    if (grown == NULL) {
        free_all(records, 200);
        return 1;
    }
    records = grown;
    for (i = 200; i < 400; i++)
        init_record(&records[i]);
    /* 4. free everything */
    free_all(records, 400);
    return 0;
```

`records = grown` comes after the `NULL` test: if `realloc` fails, the old block is still allocated, and `records` still reaches it.

After step 2, blocks on the heap: **201** bytes asked for: **23200** $= 200 \times 16 + 200 \times 100$ After step 4: **0**

The Heap After Step 3

Draw `records` and everything it reaches:



- Blocks on the heap: **401**; bytes asked for: **46400** $= 400 \times 16 + 400 \times 100$
- Calls of `free` in step 4: **401**
- `records.c` prints, on this Mac:

```
records[199]: To be set -1 0.0
the block moved
records[399]: To be set -1 0.0
```

Whether the block moves depends on the allocator; the 200 records in it are copied either way.

`leaks -atExit - ./records`:

```
0 leaks for 0 total leaked bytes
```

Summary & What's Next

Today

- **The problem**: a local array dies with its AR; the second call of `read_roster` built its AR over the first roster
- **The heap**: memory the program asks for by size while it runs and keeps until it frees it; it sits between the global data and the run-time stack, and the two grow toward each other
- **malloc and free**: `(student *)malloc(n * sizeof(student))`, `NULL` on failure, `free` returns the whole block; memory leak, double free, dangling pointer, segmentation fault
- **calloc** zeroes the block; **realloc** resizes it, copies the contents to a new block when it must move, and frees the old one
- **200 records**: one block of structs plus one block per NAME, grown to 400 with `realloc`, freed NAMEs first; `leaks` reports 0

Next lecture: linked lists.

Code from today: `roster_stack.c`, `roster_heap.c` (skeleton `roster_heap_skel.c`; both read `roster.txt` and `roster_b.txt`), `malloc_int.c`, `leak.c`, `double_free.c`, `after_free.c`, `calloc_zero.c`, `realloc_move.c`, `records.c` (skeleton `records_skel.c`), `ftranspose_heap.c` (reads `in_matrix.txt`).